

1382. Balance a Binary Search Tree

Solved 

Medium

 Topics

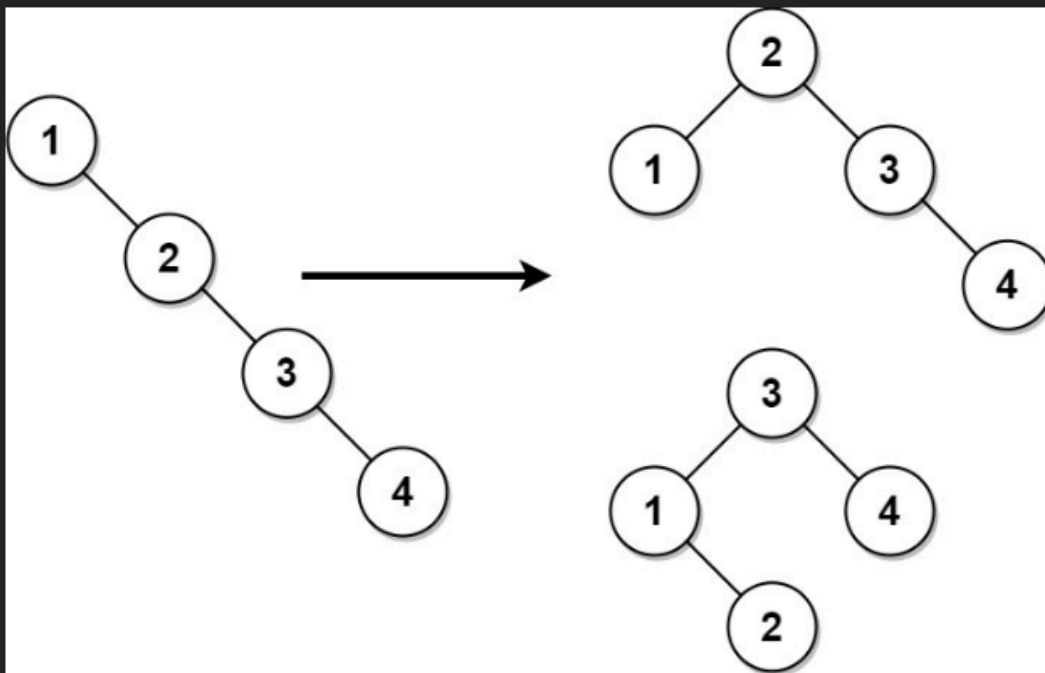
 Companies

 Hint

Given the `root` of a binary search tree, return *a **balanced** binary search tree with the same node values*. If there is more than one answer, return **any of them**.

A binary search tree is **balanced** if the depth of the two subtrees of every node never differs by more than `1`.

Example 1:

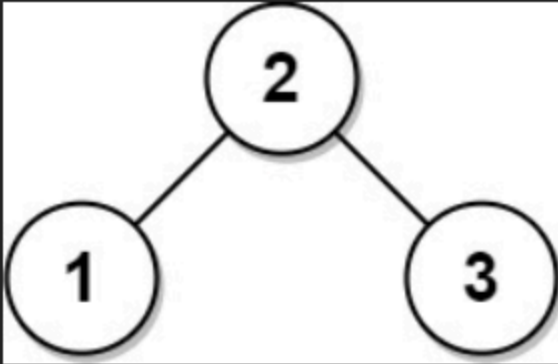


Input: `root = [1,null,2,null,3,null,4,null,null]`

Output: `[2,1,3,null,null,null,4]`

Explanation: This is not the only correct answer, `[3,1,4,null,2]` is also correct.

Example 2:



Input: root = [2,1,3]

Output: [2,1,3]

Constraints:

- The number of nodes in the tree is in the range $[1, 10^4]$.
- $1 \leq \text{Node.val} \leq 10^5$

Python:

class Solution:

```
def inorder(self, node, vals):  
    if not node:  
        return  
    self.inorder(node.left, vals)  
    vals.append(node.val)  
    self.inorder(node.right, vals)
```

```
def build(self, vals, l, r):  
    if l > r:  
        return None  
    mid = (l + r) // 2  
    node = TreeNode(vals[mid])  
    node.left = self.build(vals, l, mid - 1)  
    node.right = self.build(vals, mid + 1, r)  
    return node
```

```

def balanceBST(self, root):
    vals = []
    self.inorder(root, vals)
    return self.build(vals, 0, len(vals) - 1)

```

JavaScript:

```

/**
 * @param {TreeNode} root
 * @param {TreeNode[]} nodes
 */
var inorder = function (root, nodes) {
    if (root === null) return;
    inorder(root.left, nodes);
    nodes.push(root);
    inorder(root.right, nodes);
}

/**
 * @param {TreeNode[]} nodes
 * @param {number} start
 * @param {number} end
 * @returns {TreeNode|null}
 */
var solve = function (nodes, start, end) {
    if (end < start) return null;
    const mid = Math.floor((end - start) / 2) + start;
    const root = nodes[mid];
    root.left = solve(nodes, start, mid - 1);
    root.right = solve(nodes, mid + 1, end);
    return root;
}

/**
 * @param {TreeNode} root
 * @returns {TreeNode|null}
 */
var balanceBST = function (root) {
    if (root === null) return null;
    const inorderTraversal = [];
    inorder(root, inorderTraversal);
    return solve(inorderTraversal, 0, inorderTraversal.length - 1);
}

```

Java:

```

class Solution {

```

```

public void inorder(TreeNode node, List<Integer> vals) {
    if (node == null) return;
    inorder(node.left, vals);
    vals.add(node.val);
    inorder(node.right, vals);
}

public TreeNode build(List<Integer> vals, int l, int r) {
    if (l > r) return null;
    int mid = (l + r) / 2;
    TreeNode node = new TreeNode(vals.get(mid));
    node.left = build(vals, l, mid - 1);
    node.right = build(vals, mid + 1, r);
    return node;
}

public TreeNode balanceBST(TreeNode root) {
    List<Integer> vals = new ArrayList<>();
    inorder(root, vals);
    return build(vals, 0, vals.size() - 1);
}
}

```